Building an Equal-Weight Portfolio

Invest equal amounts in four stocks. Work through shares, cash, rebalancing and costs, then examine their joint variation. This week covers equal weighting only.

No.	Question on this page	Page
1	Does equal weighting mean equal share counts?	3
2	How do four stocks' gains and losses form a portfolio return?	4
3	Why do weights change without trading?	5
4	Where does leftover cash go when only whole shares are allowed?	6
5	When do monthly rebalanced holdings start earning returns?	7
6	How much trading restores 25% weights?	8
7	Which purchase and sale amounts incur costs?	9
8	How do we reserve costs for the first purchase?	10
9	How can rebalancing pay costs on both purchases and sales?	11
10	Would investing USD 0.01 more exceed the budget?	12
11	How do sale proceeds pay for purchases and costs?	13
12	How many shares must be bought or sold to restore equal weights?	14
13	Do we value holdings or change shares first each day?	15
14	How much remains after half a year of monthly rebalancing with costs?	16
15	Does more frequent rebalancing leave more terminal wealth?	17
16	Do equal investments have equally variable gains and losses?	18
17	How does the account differ when both stocks fall or move oppositely?	19
18	How does joint variation enter portfolio volatility?	20
19	Why does adding AAPL increase volatility here?	21

Prepare common prices for four stocks

AAPL, MSFT, JPM and XOM: 2024/1/2 to 6/28. Prices are the provider's Close (USD/share), not total returns including dividends. Stocks and schedules are teaching choices, not records of historical investment decisions.

Before running: Do all four stocks have prices on the same first and last dates?

```
import numpy as np
import pandas as pd
import yfinance as yf
tickers = ["AAPL", "MSFT", "JPM", "XOM"]
history = {}
for ticker in tickers:
    h = yf.Ticker(ticker).history(start="2023-12-29",
        end="2024-07-01", interval="1d", auto_adjust=False, actions=True)
    h.index = h.index.strftime("%Y-%m-%d")
    assert h.index.is_unique and h.index.is_monotonic_increasing
    assert h["Stock Splits"].eq(0).all()
    history[ticker] = h
all_close = pd.DataFrame({t: h["Close"] for t, h in history.items()})
assert np.isfinite(all_close).all().all() and all_close.gt(0).all().all()
close = all_close.loc["2024-01-02":].copy()
returns = close.pct_change(fill_method=None).dropna()
shown = close.iloc[[0, -1]].round(2)
shown.insert(0, "Observation date", shown.index)
shown
```

Observation date	AAPL	MSFT	JPM	XOM
2024-01-02	185.64	370.87	172.08	102.36
2024-06-28	210.62	446.95	202.26	115.12

Each stock has 124 common prices and 123 daily returns. Assume initial capital of USD 1,000, fractional shares and ideal Close fills. Enter on 1/2 and rebalance at the first observed close each month. Do not rebalance or liquidate at the end. Exclude dividends, taxes and cash interest.

Issues 1 through 6 have no costs; 7 through 15 charge an assumed 4bp on each purchase and sale amount. Week 4 used a fixed dollar cost per trade; Week 6 used a linear approximation based on opening wealth. Here the share-and-cash account pays proportional costs exactly. Issues 16 through 19 examine volatility using the same daily returns, separately from the net account.

ISSUE 01

Does equal weighting mean equal share counts?

Equal weighting divides investment amounts equally, not share counts. The table converts USD 250 per stock into shares: a higher price buys fewer shares with the same amount.

N is the number of stocks and w each target weight; W is account wealth (USD), C the Close (USD/share), and q the share count. Subscript i identifies a stock. This issue invests all capital without costs.

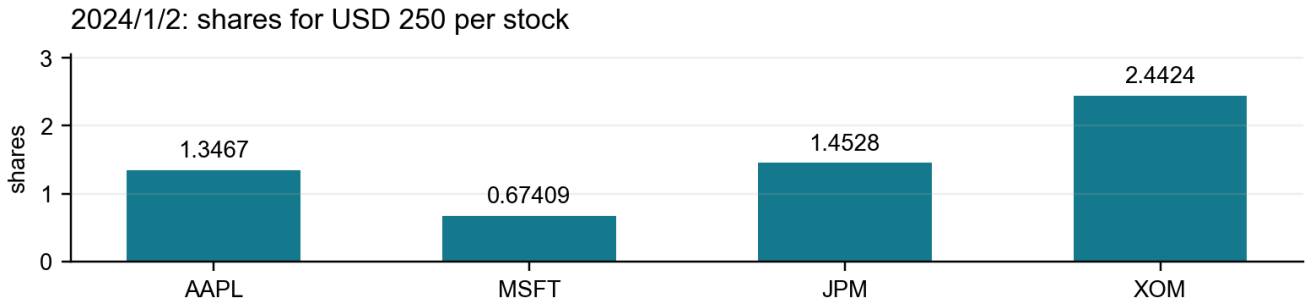
$$w_i = 1/N, \qquad q_i = w_i W / C_i$$

Before running: For the same USD 250, will MSFT require fewer shares than XOM?

```
capital = 1000.0
weight = 1 / len(tickers)
shares = capital * weight / close.iloc[0]
pd.DataFrame({"Ticker": tickers, "Price (USD/share)": close.iloc[0].round(2),
    "Invested (USD)": capital * weight, "Shares": shares.round(5)})
```

Ticker	Price (USD/share)	Invested (USD)	Shares
AAPL	185.64	250.0	1.34669
MSFT	370.87	250.0	0.67409
JPM	172.08	250.0	1.45281
XOM	102.36	250.0	2.44236

$$q_{MSFT} \approx 250/370.87000 \approx 0.67409 \text{ shares}$$



MSFT requires about 0.67409 shares and XOM 2.44236 shares, but both investments are USD 250, or 25%.

Discussion

At the same price, approximately how many MSFT shares would USD 500 per stock buy?

Sources: [1], [2], [3].

ISSUE 02

How do four stocks' gains and losses form a portfolio return?

Sum the four stocks' dollar gains and losses, then divide by opening wealth of USD 1,000 for the account return. Weighting each stock return by its opening weight gives the same result.

r_i is a stock's decimal return and r_p the portfolio's decimal return. Prices 0 and 1 are the 1/2 and 1/3 closes; here w_i=0.25. Shares remain unchanged, with no costs or cash flows. Contributions are in percentage points.

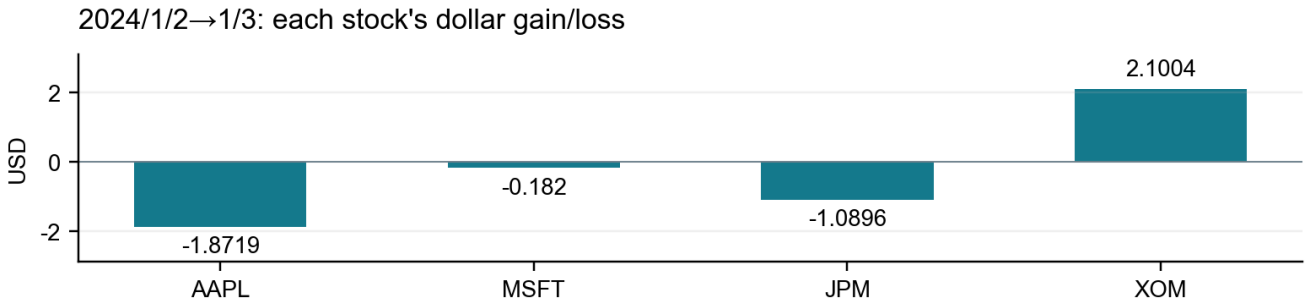
$$r_p = \sum_i w_i r_i = \frac{\sum_i q_i (C_{i,1} - C_{i,0})}{1000}$$

Before running: Is the portfolio return the sum or the average of the four percentages?

```
first_return = returns.iloc[0]
profit = shares * (close.iloc[1] - close.iloc[0])
pd.DataFrame({"Ticker": tickers, "Stock return (%)": (first_return*100).round(4),
    "Gain/loss (USD)": profit.round(4),
    "Contribution (percentage points)": (weight*first_return*100).round(4)})
```

Ticker	Stock return (%)	Gain/loss (USD)	Contribution (percentage points)
AAPL	-0.7488	-1.8719	-0.1872
MSFT	-0.0728	-0.182	-0.0182
JPM	-0.4358	-1.0896	-0.109
XOM	0.8402	2.1004	0.21

$$r_p \approx -1.043077/1000 \approx -0.1043\%$$



The four stocks total USD -1.0431, leaving account wealth of USD 998.9569. AAPL's decline does not apply to the entire account.

Discussion

Holding the other three stocks' gains/losses fixed, what minimum XOM dollar gain would prevent an account loss that day?

Sources: [1], [2].

ISSUE 03

Why do weights change without trading?

Weights can move away from 25% without trading. Shares stay unchanged; stocks whose prices rise relatively more take a larger account share. The table divides by total wealth on 1/31.

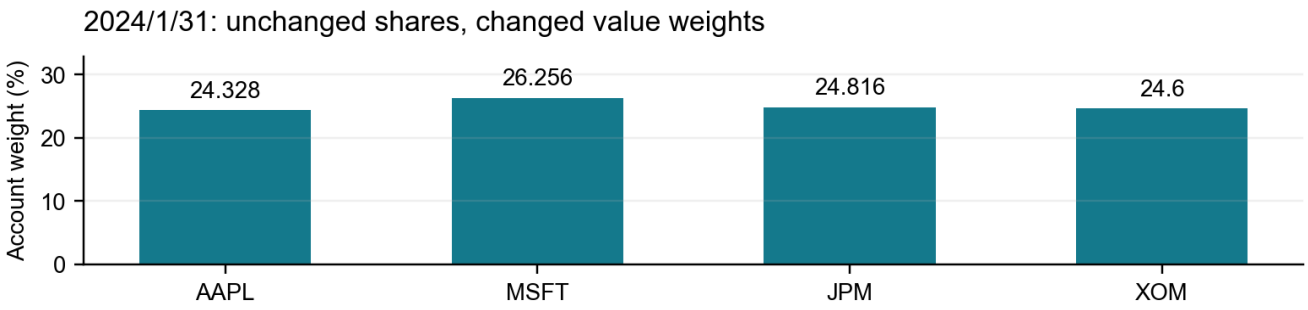
$V_i=q_iC_i$ is a stock's dollar value (shares×today's Close), W total dollar wealth, and $w_i=V_i/W$. Hold from 1/2 to 1/31 without trading in this issue.

Before running: Are all four weights still 25% on 1/31?

```
jan_values = shares * close.loc["2024-01-31"]
jan_weights = jan_values / jan_values.sum()
pd.DataFrame({"Ticker": tickers, "Initial (%)": 25.0,
    "1/31 value (USD)": jan_values.round(5),
    "1/31 weight (%)": (100*jan_weights).round(2)})
```

Ticker	Initial (%)	1/31 value (USD)	1/31 weight (%)
AAPL	25.0	248.33009	24.33
MSFT	25.0	268.00496	26.26
JPM	25.0	253.31241	24.82
XOM	25.0	251.09905	24.6

$$w_{AAPL} \approx 248.33009/1020.74652 \approx 24.33\%$$



AAPL falls to 24.33% and MSFT rises to 26.26%. Restoring 25% next time requires trades based on current values.

Discussion

With no costs or added capital on 1/31, how many dollars of MSFT should be bought or sold to restore 25%?

Sources: [1], [2].

ISSUE 04

Where does leftover cash go when only whole shares are allowed?

Give each stock only USD 250 and round shares down. Buy nothing if the budget is below one share. Unused funds remain cash, so stock weights do not sum to 100%.

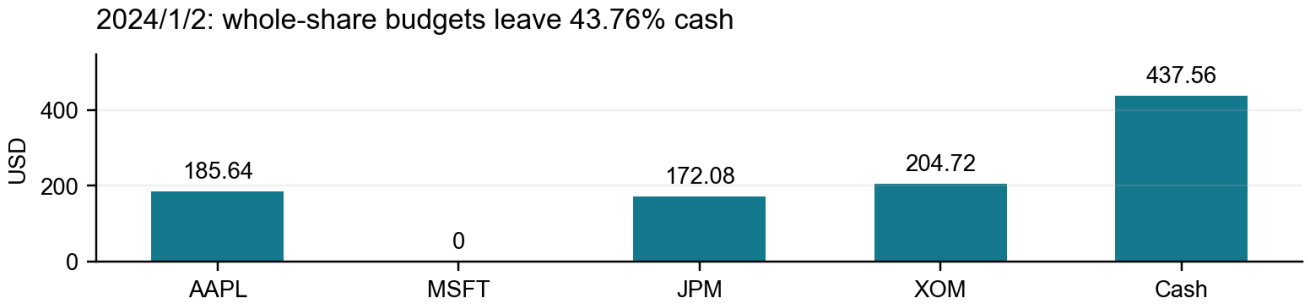
floor rounds down, not to the nearest integer. With no costs, this issue only truncates shares under each stock's budget; it does not search for the closest equal-weight allocation. Later issues return to fractional shares.

Before running: Can USD 250 buy one MSFT share? Does leftover cash still count as an account asset?

```
whole_shares = np.floor(250 / close.iloc[0])
whole_values = whole_shares * close.iloc[0]
cash_left = capital - whole_values.sum()
positions = whole_values.copy()
positions.loc["Cash"] = cash_left
counts = whole_shares.astype(int).astype(str)
pd.DataFrame({"Holding": positions.index,
    "Shares": counts.reindex(positions.index, fill_value="-"),
    "Value (USD)": positions.round(2),
    "Weight (%)": (positions/capital*100).round(2)})
```

Holding	Shares	Value (USD)	Weight (%)
AAPL	1	185.64	18.56
MSFT	0	0.0	0.0
JPM	1	172.08	17.21
XOM	2	204.72	20.47
Cash	-	437.56	43.76

Share counts are 1, 0, 1 and 2, leaving about USD 437.56 cash, or 43.76%. Rescaling stock weights alone to 100% hides the cash.



Truncating shares spends only USD 562.44. The remaining USD 437.56 has no stock gains/losses and earns no interest.

Discussion

With one-quarter of the budget per stock, approximately what minimum total capital can buy one MSFT share?

Sources: [1], [3].

ISSUE 05

When do monthly rebalanced holdings start earning returns?

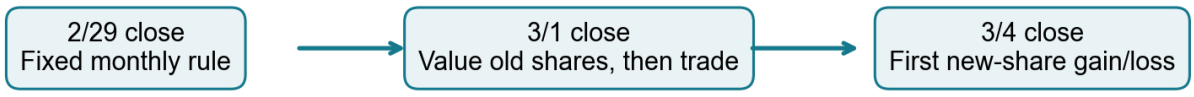
Rebalancing must change shares. Fix the monthly rule in advance: mark the previous close as the decision date and trade at the month's first observed close. New shares earn returns only from the next interval.

Before running: Can shares traded at the 3/1 close earn the return already completed by 3/1?

```
dates = all_close.index
months = pd.to_datetime(close.index).to_period("M")
monthly_dates = close.index[~months.duplicated()]
timing = []
for date in monthly_dates:
    j = dates.get_loc(date)
    timing.append([dates[j-1], date, dates[j+1]])
pd.DataFrame(timing, columns=["Prior decision date", "Close-fill date", "First new-share return end"])
```

Prior decision date	Close-fill date	First new-share return end
2023-12-29	2024-01-02	2024-01-03
2024-01-31	2024-02-01	2024-02-02
2024-02-29	2024-03-01	2024-03-04
2024-03-28	2024-04-01	2024-04-02
2024-04-30	2024-05-01	2024-05-02
2024-05-31	2024-06-03	2024-06-04

2024/3: separate old and new shares



Old shares bear the change ending on 3/1. New shares earn only 3/1→3/4.

Assume dollar allocations can buy fractional shares at the execution-day close; share counts are determined then. This is an ideal rule, not an actual brokerage service or trade record.

Discussion

After changing shares at the 4/1 close, when does the first return on the new holdings end?

Sources: [1], [3].

ISSUE 06

How much trading restores 25% weights?

To restore 25% on 2/1, first take one-quarter of current wealth. Sell positions above it and buy those below it; do not compare against the original USD 250.

V_i is each stock's pre-trade value, W total wealth, and ΔV_i the trade amount, all in USD. Positive means buy; negative means sell. This issue has no costs.

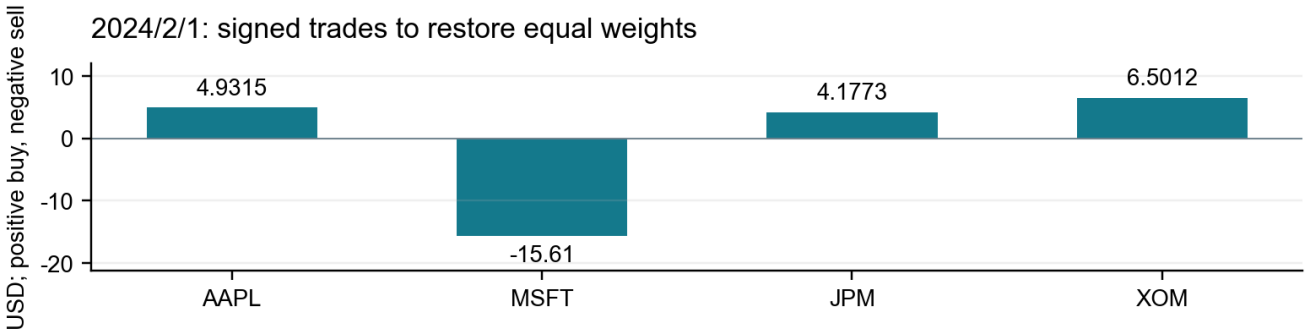
$$W = \sum_i V_i, \qquad \Delta V_i = W/4 - V_i$$

Before running: With no costs, will total purchases equal total sales?

```
pre_values = shares * close.loc["2024-02-01"]
pre_wealth = pre_values.sum()
target_value = pre_wealth / len(tickers)
trade = target_value - pre_values
pd.DataFrame({"Ticker": tickers, "Before trade (USD)": pre_values.round(2),
    "Target (USD)": round(target_value, 2),
    "Buy/sell (USD)": trade.round(2)})
```

Ticker	Before trade (USD)	Target (USD)	Buy/sell (USD)
AAPL	251.64	256.57	4.93
MSFT	272.18	256.57	-15.61
JPM	252.4	256.57	4.18
XOM	250.07	256.57	6.5

$$\Delta V_{AAPL} \approx 1026.29769/4 - 251.64297 \approx 4.93146 \text{ USD}$$



Buy about USD 4.93 of AAPL and sell USD 15.61 of MSFT. Total purchases and sales are each 15.61; trading itself creates no profit.

Discussion

Approximately how many shares does an AAPL purchase of USD 4.93 at USD 186.86/share add?

Sources: [1], [2], [4].

ISSUE 07

Which purchase and sale amounts incur costs?

Charge costs on both purchase and sale amounts in USD, not on merely held positions. First price the zero-cost orders; the next issue funds those costs from the account.

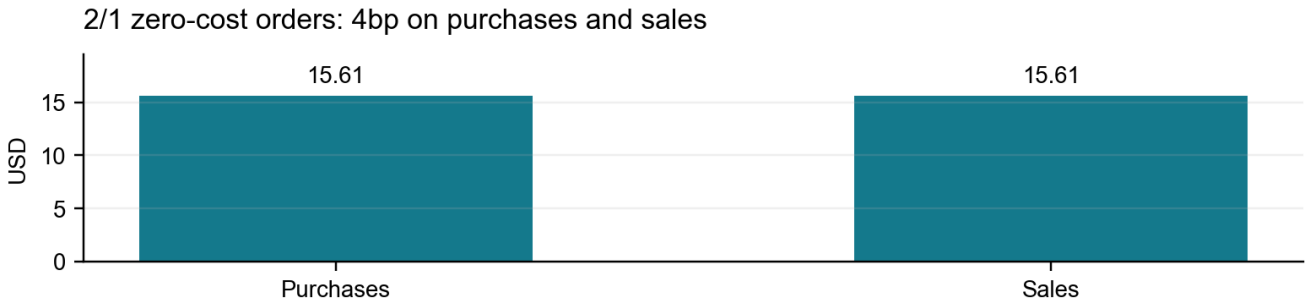
Q is purchases plus sales (USD), c the decimal cost rate, and F cost (USD). Lowercase q remains shares, distinct from Q. Here c=4bp=4/10000=0.0004.

$$Q = \sum_i |\Delta V_i|, \qquad F = cQ$$

Before running: Would charging only purchases on 2/1 omit half the traded amount?

```
bought = trade.clip(lower=0).sum()
sold = -trade.clip(upper=0).sum()
pd.DataFrame({"Event": ["Initial all-buy order", "2/1 rebalance"],
    "Purchases (USD)": [capital, round(bought, 4)],
    "Sales (USD)": [0, round(sold, 4)],
    "Cost (USD)": [capital*.0004, round((bought+sold)*.0004, 4)]})
```

Event	Purchases (USD)	Sales (USD)	Cost (USD)
Initial all-buy order	1000.0	0.0	0.4
2/1 rebalance	15.6099	15.6099	0.0125



Each side is USD 15.6099, totaling 31.2198. Do not charge 4bp on all USD 1026.30.

Spending all 1,000 initially and then charging 0.40 exceeds the budget. This table only prices zero-cost orders; purchases have not yet been reduced to pay costs.

Discussion

Someone charges 4bp on all USD 1026.30. Which amount should bear costs here?

Sources: [1], [5].

ISSUE 08

How do we reserve costs for the first purchase?

At initial entry, buy slightly less to pay costs; do not spend 1,000 and borrow the cost. Equal weights divide the amount invested after paying costs.

W is the original USD 1,000 account; I is total stock investment, not cash B; c=0.0004. Initial trades are all purchases, so cost F=cI.

$$I + cI = W \quad \Longrightarrow \quad I = W/(1 + c)$$

Before running: After reducing actual purchases, will cost still be exactly USD 0.40?

```
entry_budget = capital / (1 + .0004)
entry_fee = .0004 * entry_budget
pd.DataFrame({"Initial cash (USD)": [capital],
    "Investment I (USD)": [round(entry_budget, 5)],
    "Transaction cost (USD)": [round(entry_fee, 5)],
    "Investment + cost": [round(entry_budget+entry_fee, 5)]})
```

Initial cash (USD)	Investment I (USD)	Transaction cost (USD)	Investment + cost
1000.0	999.60016	0.39984	1000.0

$$I = 1000/1.0004 \approx 999.60016, \quad F \approx 0.39984 \text{ USD}$$

Initial entry: divide the amount after costs



Investment plus cost equals exactly 1,000. Each stock receives about USD 249.90004, without borrowing.

Discussion

If you reserve 0.40 and buy USD 999.60, how much cash remains after costs on actual purchases?

Sources: [5].

ISSUE 09

How can rebalancing pay costs on both purchases and sales?

Continue the same account after paying its entry cost on 1/2. On 2/1, value the existing shares before setting new stock amounts. Sale proceeds must pay for both purchases and costs.

V_i is each old stock position's dollar value, $W=\Sigma V_i$, and starting cash B=0, without borrowing. I is investment after costs; each new target is I/4. Costs use the sum of absolute target-minus-old values. Post-trade stocks total I, so I + cost = W.

The 1/2 investment was USD 999.60016, or USD 249.90004 per stock. Value those shares on 2/1 below. First try investing USD 1025.87. Changing the target also changes trade amounts and costs.

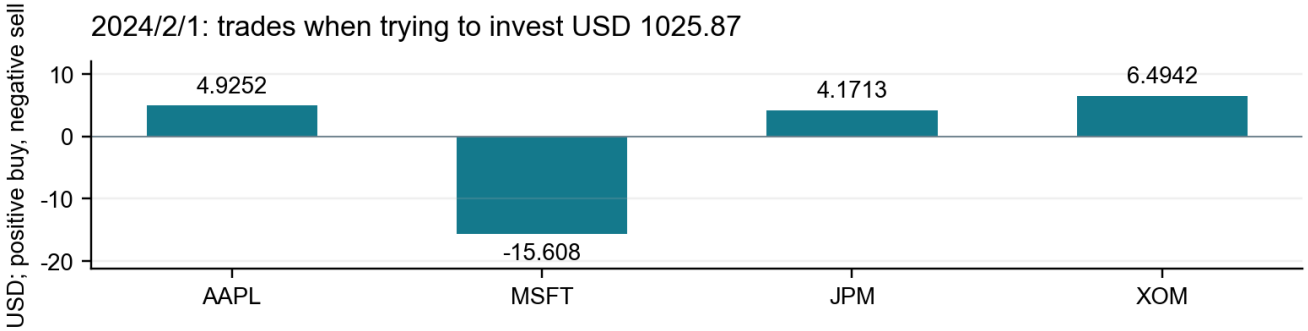
$$I + c\sum_i |I/4 - V_i| = W$$

Before running: Will all MSFT sale proceeds buy the other three stocks?

```
entry_q = entry_budget / 4 / close.iloc[0]
feb_values = entry_q * close.loc["2024-02-01"]
feb_before = feb_values.sum()
trial = 1025.87
trial_trade = trial / 4 - feb_values
trial_fee = .0004 * abs(trial_trade).sum()
pd.DataFrame({"Ticker": tickers, "Old value (USD)": feb_values.round(4),
    "Target (USD)": round(trial/4, 4),
    "Trade amount (USD)": trial_trade.round(4)})
```

Ticker	Old value (USD)	Target (USD)	Trade amount (USD)
AAPL	251.5423	256.4675	4.9252
MSFT	272.0755	256.4675	-15.608
JPM	252.2962	256.4675	4.1713
XOM	249.9733	256.4675	6.4942

$$1025.87000 + 0.012479 \approx 1025.882479 < 1025.887338$$



Investment plus costs leaves USD 0.004858. Not all sale proceeds go into stock purchases.

Discussion

If all sale proceeds bought stocks, how would this account with no cash pay its costs?

Sources: [5].

ISSUE 10

Would investing USD 0.01 more exceed the budget?

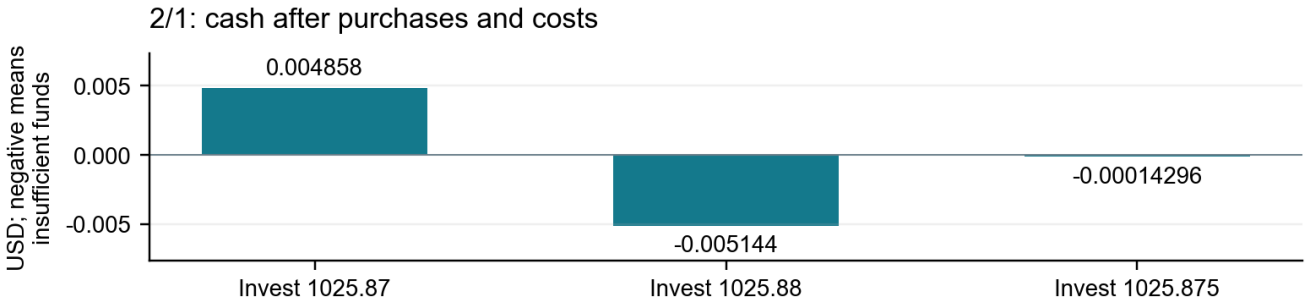
Use the same 2/1 account with USD 1025.88734 available. Compare stock investments differing by just USD 0.01 to check whether cash covers costs.

Before running: After buying USD 1025.88 of stocks, can the remaining funds pay the costs?

```
trials = []
for proposed in [1025.87, 1025.88, 1025.875]:
    fee = .0004 * abs(proposed/4 - feb_values).sum()
    trials.append([proposed, fee, proposed+fee,
                   feb_before-proposed-fee])
pd.DataFrame(trials, columns=["Invested (USD)", "Cost (USD)",
    "Total spending (USD)", "Remaining (USD)"]).round(6)
```

Invested (USD)	Cost (USD)	Total spending (USD)	Remaining (USD)
1025.87	0.012479	1025.882479	0.004858
1025.88	0.012481	1025.892481	-0.005144
1025.875	0.01248	1025.88748	-0.000143

Investing 1025.87 leaves cash; 1025.88 overspends. The midpoint 1025.875 also slightly overspends. Invest a little less to avoid negative cash. The next issue finds the amount that leaves exactly enough for costs.



This ideal proportional-cost example does not round costs to cents. Extra decimals show whether the budget is exceeded.

Discussion

If a small cash balance is allowed, which investment in the table is already affordable?

Sources: [5].

ISSUE 11

How do sale proceeds pay for purchases and costs?

Split MSFT sale proceeds between purchases of the other three stocks and trading costs. The function bisects a range to find the investment after costs. Overspending lowers the upper bound; spare cash raises the lower bound.

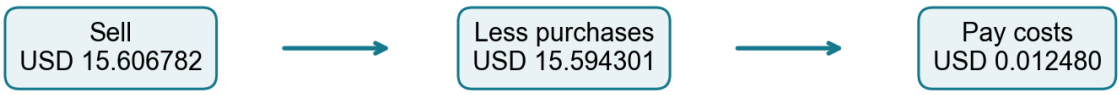
Before running: Will sales minus purchases equal this trade's costs?

```
def investable_value(values, cash, rate):
    available = values.sum() + cash
    low, high = 0.0, available
    for _ in range(50):
        budget = (low + high) / 2
        fee = rate * abs(budget/len(values) - values).sum()
        if budget + fee > available:  # Overspending: lower the upper bound to the midpoint
            high = budget
        else:  # Affordable: raise the lower bound to the midpoint
            low = budget
    fee = rate * abs(low/len(values) - values).sum()
    return low, fee  # low always remains an affordable investment
feb_I, feb_F = investable_value(feb_values, 0, .0004)
feb_trade = feb_I / 4 - feb_values
feb_buy = feb_trade.clip(lower=0).sum()
feb_sell = -feb_trade.clip(upper=0).sum()
pd.DataFrame({"Sale proceeds (USD)": [feb_sell], "Purchase amount (USD)": [feb_buy],
    "Transaction cost (USD)": [feb_F], "Cash after trading": [feb_sell-feb_buy-feb_F]}).round(6)
```

Sale proceeds (USD)	Purchase amount (USD)	Transaction cost (USD)	Cash after trading
15.606782	15.594301	0.01248	0.0

On 2/1, investment of 1025.87486 plus costs of USD 0.012480 equals the pre-trade 1025.88734. Each new stock value is USD 256.46871.

2/1: sales minus purchases leave the costs



Starting cash is 0. Sale proceeds partly buy stocks and partly pay costs, without borrowing.

Here, investing more increases total spending including costs, so bisection narrows the range. The 50 iterations set numerical precision; they are not trades.

Discussion

Using the table, would cash still suffice if purchases increased by USD 0.01 without additional sales?

Sources: [5].

ISSUE 12

How many shares must be bought or sold to restore equal weights?

The previous issue left USD 256.46871 per stock after costs. Divide by each 2/1 price for the new share count. Subtract existing shares to find shares bought or sold.

q is shares, C the 2/1 Close (USD/share), and I the previous issue's total stock value after costs. New shares=(I/4)/C. A positive share change means a purchase; a negative change means a sale.

Before running: Does restoring the same 25% still require different share counts across stocks?

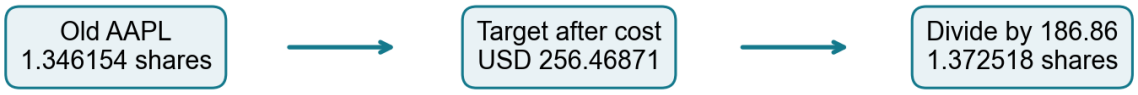
```
feb_q = feb_I / 4 / close.loc["2024-02-01"]
pd.DataFrame({"Ticker": tickers, "2/1 price": close.loc["2024-02-01"].round(2),
    "Old shares": entry_q.round(6), "New shares": feb_q.round(6),
    "Shares bought/sold": (feb_q-entry_q).round(6)})
```

Ticker	2/1 price	Old shares	New shares	Shares bought/sold
AAPL	186.86	1.346154	1.372518	0.026364
MSFT	403.78	0.673821	0.635169	-0.038652
JPM	173.73	1.452232	1.476249	0.024017
XOM	102.39	2.441384	2.504822	0.063438

$$q_{AAPL,\,new} \approx (1025.87486/4)/186.86 \approx 1.372518 \text{ shares}$$

Buy about 0.026364 AAPL shares and sell 0.038652 MSFT shares. New share counts differ, but each count times its price equals USD 256.46871.

2/1: value old shares, pay costs, then calculate new shares



AAPL increases from 1.346154 to 1.372518 shares. The new shares bear price changes only after the 2/1 close.

Discussion

Multiply AAPL shares purchased by USD 186.86/share. Approximately how much is spent on the purchase?

Sources: [1], [5].

ISSUE 13

Do we value holdings or change shares first each day?

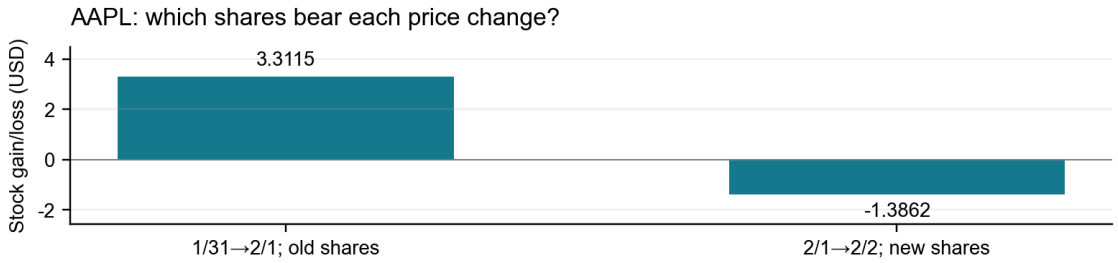
From 1/31 to 2/1, value 1.346154 AAPL shares. Change to 1.372518 only after the 2/1 trade. There is no trade on 2/2, so continue valuing the new shares. Apply the same order to the other three stocks.

Before running: With the same price before and after the 2/1 trade, does AAPL's value rise because of a price gain or extra shares purchased?

```
steps = []
for date, stage in [("2024-01-31", " close"), ("2024-02-01", " before trade"),
                    ("2024-02-01", " after trade"), ("2024-02-02", " close")]:
    q = entry_q if date == "2024-01-31" or stage == " before trade" else feb_q
    values = q * close.loc[date]
    steps.append([date[5:]+stage, round(close.loc[date, "AAPL"], 2),
                  round(q.AAPL, 6), round(values.AAPL, 4), round(values.sum(), 5)])
pd.DataFrame(steps, columns=["Time", "AAPL price", "AAPL shares",
    "AAPL value", "Account wealth"])
```

Time	AAPL price	AAPL shares	AAPL value	Account wealth
01-31 close	184.4	1.346154	248.2308	1020.33838
02-01 before trade	186.86	1.346154	251.5423	1025.88734
02-01 after trade	186.86	1.372518	256.4687	1025.87486
02-02 close	185.85	1.372518	255.0825	1029.63851

Table prices are USD/share; stock values and wealth are USD. The price is unchanged across the 2/1 trade. Extra shares increase AAPL's value, while account wealth falls only by the USD 0.012480 cost.



Old shares earn USD +3.3115; new shares bear USD -1.3862. Purchasing extra shares is not itself a gain.

Discussion

Why would treating AAPL's value increase across the 2/1 trade as a gain be wrong?

Sources: [1], [5].

ISSUE 14

How much remains after half a year of monthly rebalancing with costs?

Apply the previous order over 1/2 to 6/28. Value holdings daily; rebalance and pay costs at each month's first observed close. Start with the original USD 1,000 and calculate the amount left after half a year.

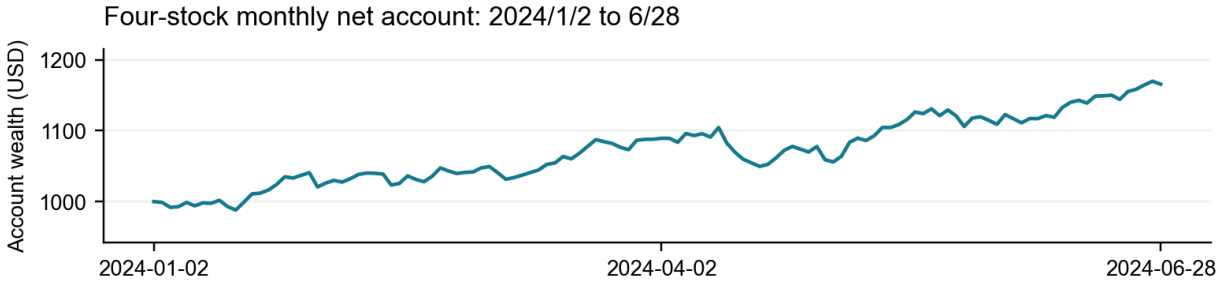
Before running: Should the full-period return divide by the original 1,000 or USD 999.60016 after entry costs?

```
def run_portfolio(rebalance_dates, rate):
    q = pd.Series(0.0, index=tickers)
    cash = capital
    rows = []
    for date, prices in close.iterrows():
        values = q * prices
        available = values.sum() + cash
        fee = 0.0
        if date in rebalance_dates:
            budget, fee = investable_value(values, cash, rate)
            q = budget / len(tickers) / prices
            cash = available - budget - fee
        wealth = (q * prices).sum() + cash
        rows.append([date, wealth, fee])
    return pd.DataFrame(rows, columns=["Date", "Wealth", "Fee"]).set_index("Date")
monthly = run_portfolio(monthly_dates, .0004)
shown = monthly.loc[["2024-01-02", "2024-02-01", "2024-06-28"]].round(5)
shown.columns = ["Account wealth (USD)", "Cost (USD)"]
shown.insert(0, "Observation date", shown.index)
shown
```

Observation date	Account wealth (USD)	Cost (USD)
2024-01-02	999.60016	0.39984
2024-02-01	1025.87486	0.01248
2024-06-28	1165.40253	0.0

r is the full-period account return relative to original capital. The USD 1,000 denominator retains the effect of entry costs.

$$r \approx 1165.40253/1000 - 1 \approx 16.5403\%$$



USD 1,000 becomes about USD 1,165.40 after every trading cost. The endpoint values holdings without deducting liquidation costs.

Discussion

If all stocks were liquidated on 6/28 with a 4bp selling cost, how many dollars would remain?

Sources: [1], [5].

ISSUE 15

Does more frequent rebalancing leave more terminal wealth?

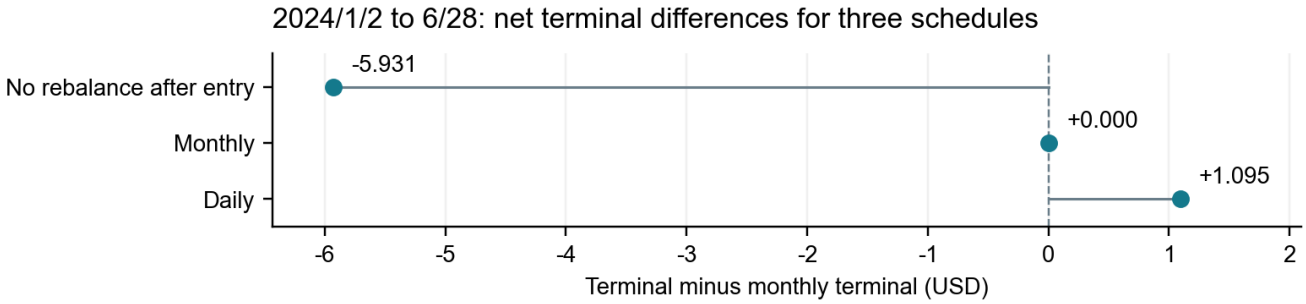
Change only rebalancing frequency. Keep the four stocks, dates, initial capital, Close fills and 4bp per direction fixed. Recompute gross and net accounts daily; do not merely subtract costs at the end.

Before running: Must the most rebalances produce the highest terminal wealth after costs?

```
schedules = {"No rebalance after entry": close.index[:1],
    "Monthly": monthly_dates, "Daily": close.index[:-1]}
comparison, net_paths = [], {}
for name, schedule in schedules.items():
    gross = run_portfolio(schedule, 0)
    net = run_portfolio(schedule, .0004)
    net_paths[name] = net["Wealth"]
    comparison.append([name, len(schedule)-1, net["Fee"].sum(),
        gross["Wealth"].iloc[-1], net["Wealth"].iloc[-1]])
comparison = pd.DataFrame(comparison, columns=["Schedule", "Rebalance count",
    "Cost (USD)", "Terminal at 0 rate", "Terminal at 4bp"])
comparison.round(3)
```

Schedule	Rebalance count	Cost (USD)	Terminal at 0 rate	Terminal at 4bp
No rebalance after entry	0	0.4	1159.936	1159.472
Monthly	5	0.476	1165.951	1165.403
Daily	122	0.807	1167.406	1166.497

Daily terminal wealth after costs exceeds monthly by about USD 1.095 (subtracting unrounded values). The monthly terminal-column difference is 0.548609, versus total cost USD 0.475722. These differ: paying costs on trade dates reduces shares and later capital. Subtracting total cost only at the end is incorrect. 4bp is assumed, not a brokerage bill.



Zero is monthly terminal wealth after costs, USD 1165.40253. Subtract at original precision, then round. All three enter on 1/2 and only value holdings on 6/28, without liquidation costs. Other periods need not show the same differences.

Discussion

With 122 daily and 5 monthly rebalances, why is total cost not 24.4 times as large?

Sources: [1], [4], [5].

ISSUE 16

Do equal investments have equally variable gains and losses?

Investing USD 250 in each stock does not make gains/losses equally variable. Fix each daily investment to isolate stock-return variation; this issue does not track the earlier monthly account.

SD means standard deviation. Use the 123 daily returns over 1/3 to 6/28; s_i is sample daily SD (decimal, ddof=1). Start each interval with USD 250 per stock, so dollar gain/loss=$250r_i$.

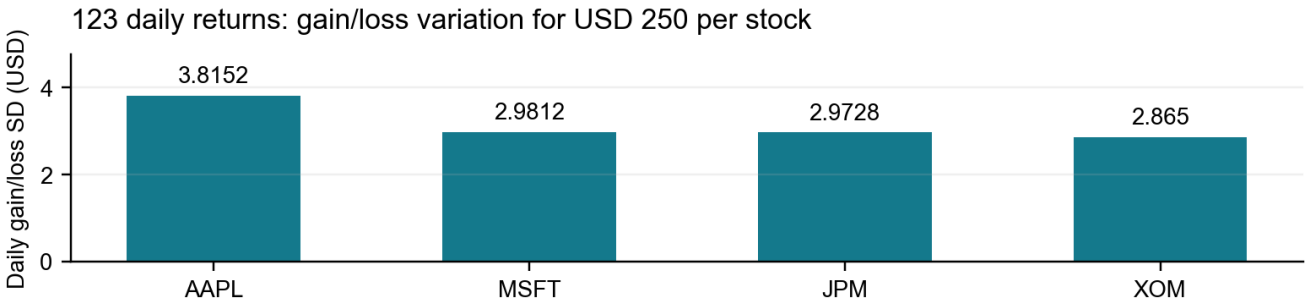
$$\mathrm{SD}(250r_i) = 250s_i$$

Before running: With USD 250 per stock, must the four gain/loss standard deviations be equal?

```
daily_sd = returns.std(ddof=1)
dollar_sd = 250 * daily_sd
pd.DataFrame({"Ticker": tickers,
    "Daily return SD (%)": (daily_sd*100).round(4),
    "USD 250 gain/loss SD (USD)": dollar_sd.round(3)})
```

Ticker	Daily return SD (%)	USD 250 gain/loss SD (USD)
AAPL	1.5261	3.815
MSFT	1.1925	2.981
JPM	1.1891	2.973
XOM	1.146	2.865

$$250s_{AAPL} \approx 250(0.01526074) \approx 3.815 \text{ USD}$$



AAPL is about 3.815 and XOM USD 2.865. These are not maximum losses; adding the four bars does not give portfolio SD.

Discussion

Using the table, approximately how many dollars larger is AAPL's daily gain/loss SD than XOM's?

Sources: [1], [4], [5].

ISSUE 17

How does the account differ when both stocks fall or move oppositely?

With half the capital in each stock, two falling stocks both lose money. Opposite moves can partly offset gains and losses. Examine actual dates before measuring full-period volatility.

Start each interval with USD 500 in MSFT and USD 500 in XOM, without costs. This daily fixed-weight comparison is separate from the monthly net account. M and X denote the stocks; decimal return $r_p=(r_M+r_X)/2$.

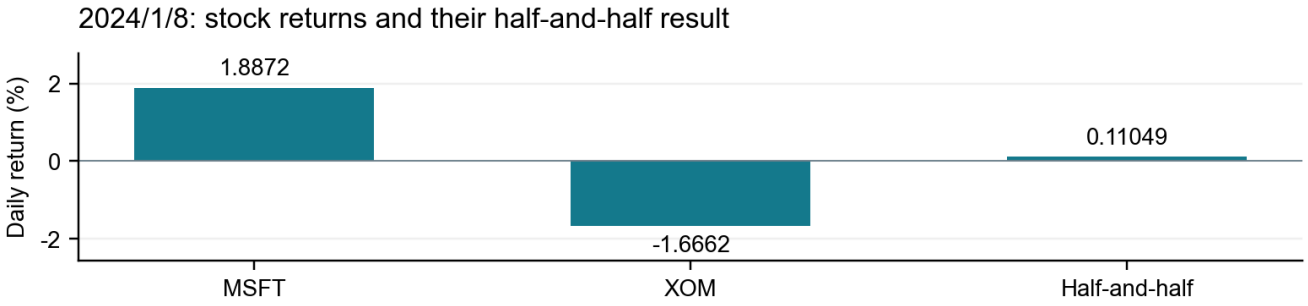
Before running: On 1/8, MSFT rises 1.8872% and XOM falls 1.6662%. Will the half-and-half portfolio also rise nearly 1.9%?

```
pair_return = returns[["MSFT", "XOM"]].mean(axis=1)
example_dates = ["2024-01-03", "2024-01-04", "2024-01-08"]
pair_examples = returns.loc[example_dates, ["MSFT", "XOM"]] * 100
pair_examples.columns = ["MSFT (%)", "XOM (%)"]
pair_examples["Half-and-half (%)"] = 100 * pair_return.loc[pair_examples.index]
pair_examples.insert(0, "Close interval", ["1/2→1/3", "1/3→1/4", "1/5→1/8"])
pair_examples.round(4)
```

Close interval	MSFT (%)	XOM (%)	Half-and-half (%)
1/2→1/3	-0.0728	0.8402	0.3837
1/3→1/4	-0.7178	-0.8719	-0.7948
1/5→1/8	1.8872	-1.6662	0.1105

$$r_{p,Jan8} \approx (1.8872\% - 1.6662\%)/2 \approx 0.1105\%$$

Both stocks fall on 1/4, and the portfolio falls about 0.7948%. Opposite moves on 1/8 produce a gain of only about 0.1105%: roughly USD 1.10 on USD 1,000. Offsetting on one day does not guarantee offsetting on other dates.



Both stock moves exceed 1%, but opposite directions leave the half-and-half portfolio with a change of only 0.1105%.

Discussion

With USD 500 in each stock on 1/4, approximately how many dollars does the account lose using the table's returns?

Sources: [1], [5].

ISSUE 18

How does joint variation enter portfolio volatility?

Opposite moves can offset, as the previous issue showed. To measure portfolio volatility over 123 intervals, include how the two returns deviate together from their own means. Do not simply average their two standard deviations.

M and X denote MSFT and XOM. d_M and d_X are each daily return minus its own mean. The half-and-half portfolio also averages their deviations.

Means over 123 intervals: MSFT 0.15889%, XOM 0.10206%. Deviations on 1/8 are 1.88715-0.15889≈1.72826 and -1.66618-0.10206≈-1.76824 percentage points. Their product is approximately -3.05599 squared percentage points: negative.

$$[(d_M + d_X)/2]^2 = d_M^2/4 + d_X^2/4 + d_M d_X/2$$

Sum the 123 deviation products and divide by 122 for sample covariance. s is daily SD of decimal returns and ρ_{MX} is correlation; covariance=$\rho_{MX} s_M s_X$. The third term of the squared expansion must therefore remain.

$$s_p^2 = s_M^2/4 + s_X^2/4 + \rho_{MX} s_M s_X/2$$

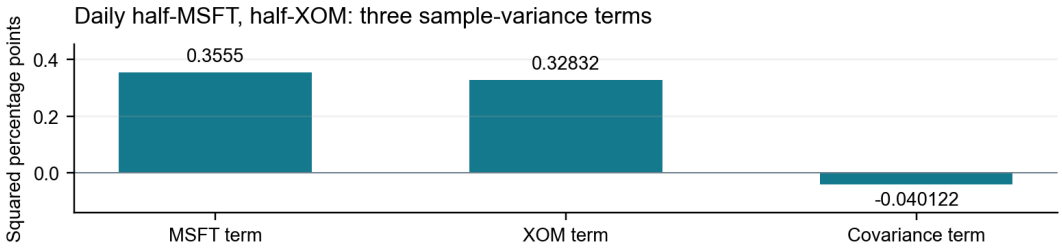
Before running: With negative correlation, does the final term increase or decrease this example's variance?

```
rho_MX = returns.MSFT.corr(returns.XOM)
pair_sd = pair_return.std(ddof=1)
pair_terms = [daily_sd.MSFT**2/4, daily_sd.XOM**2/4,
              rho_MX*daily_sd.MSFT*daily_sd.XOM/2]
pd.DataFrame({"MSFT SD ( % ) ": [round(daily_sd.MSFT*100, 4)],
    "XOM SD ( % ) ": [round(daily_sd.XOM*100, 4)],
    "ρMX": [round(rho_MX, 5)], "Portfolio SD (%)": [round(pair_sd*100, 4)]})
```

MSFT SD (%)	XOM SD (%)	ρMX	Portfolio SD (%)
1.1925	1.146	-0.05872	0.8023

The three terms below are in squared percentage points. Their square root gives daily SD in percent.

$$100 s_p \approx \sqrt{0.3555011 + 0.3283184 - 0.0401216} \approx 0.8023$$



Full-period ρMX is about -0.05872. Including the negative covariance term gives portfolio daily SD of 0.8023%.

Discussion

Would using only the first two figure terms, without the covariance term, overestimate or underestimate daily SD here?

Sources: [5].

ISSUE 19

Why does adding AAPL increase volatility here?

After adding AAPL, each stock has weight 1/3; the original pair together has 2/3. Calculate with the new weights, individual volatility and joint variation.

r_{old} is the original half-MSFT, half-XOM daily return, and r_A the AAPL daily return. New return=$(2/3)r_{old}+(1/3)r_A$. s_{old}, s_A and s_{new} are daily SDs of decimal returns. $\rho_{old,A}$ is correlation between the original portfolio and AAPL, not the previous ρ_{MX}.

$$s_{new}^2 = \frac{4}{9}s_{old}^2 + \frac{1}{9}s_A^2 + \frac{4}{9}\rho_{old,A}s_{old}s_A$$

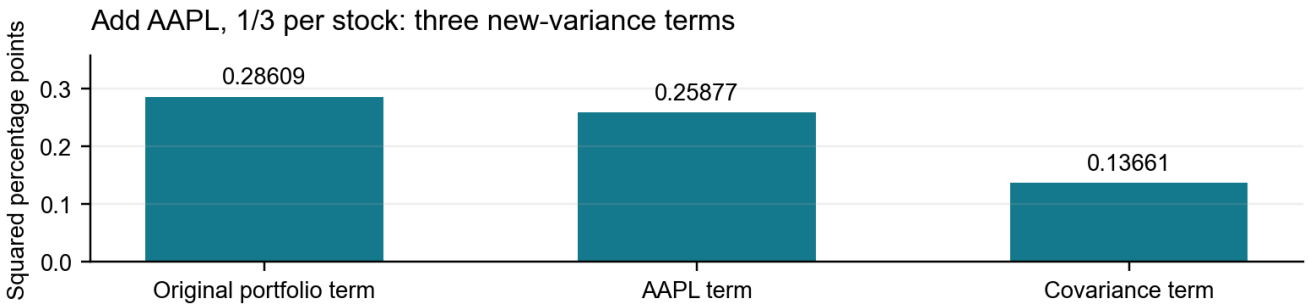
Before running: Must daily SD fall below 0.8023% after adding AAPL?

```
rho_old_A = pair_return.corr(returns.AAPL)
new_terms = [(2*pair_sd/3)**2, (daily_sd.AAPL/3)**2,
             4/9*rho_old_A*pair_sd*daily_sd.AAPL]
groups = {"MSFT + XOM": ["MSFT", "XOM"],
    "Add AAPL": ["MSFT", "XOM", "AAPL"], "Also add JPM": tickers}
volatility = pd.DataFrame([[name, len(stocks),
    100*returns[stocks].mean(axis=1).std(ddof=1)]
    for name, stocks in groups.items()],
    columns=["Equal-weight holdings", "Stock count N", "Daily SD (%)"])
volatility.round(4)
```

Equal-weight holdings	Stock count N	Daily SD (%)
MSFT + XOM	2	0.8023
Add AAPL	3	0.8255
Also add JPM	4	0.707

s_{old}=0.0080231, s_A=0.0152607, and $\rho_{old,A}$=0.25104. Multiply all three terms by 10,000 to obtain squared percentage points. The square root is $100s_{new}$; 0.8255 means daily SD is 0.8255%.

$$100s_{new} \approx \sqrt{0.2860880 + 0.2587670 + 0.1366104} \approx 0.8255$$



Sum the three terms and take the square root. The positive covariance term cannot be omitted here.

Across these 123 intervals (2024/1/3 to 6/28), adding AAPL raises sample daily SD to 0.8255%. Other periods require separate checks.

Discussion

If the positive covariance term were omitted, which direction of volatility change would you infer from adding AAPL?

Sources: [4], [5].

References and reproduction conditions

Equal-weight definition, budget and this week's data boundaries

[1] yfinance: historical price fields

[2] MSCI: Equal Weighted Indexes Methodology (2021/5), pp. 3 to 5

[3] Investor.gov: fractional-share investing

[4] Investor.gov: asset allocation, rebalancing and diversification

[5] Lobo, Fazel and Boyd (2007): proportional and fixed transaction costs, Sections 1 and 1.1

Reuse the four Yahoo/yfinance responses retained on 2026/9/18; no prices were downloaded this time. Each has 125 observations over 2023/12/29 to 2024/6/28. The account uses 124 common closes over 2024/1/2 to 6/28 and 123 daily returns. 12/29 only marks the prior decision date; do not fill or interpolate.

[2] supports the equal-weight definition. Stocks and rebalancing at each month's first observed close are teaching choices, not an index reproduction. [5] supports budget, proportional-cost and covariance relationships. This week derives its own equal-weight budget values, without teaching optimization or other weighting methods.

Close is in USD/share, not total return including dividends. Fractional shares and Close fills are ideal assumptions, not actual executions. Exclude dividends, taxes and cash interest. The 4bp cost rate is assumed, not a broker quote. Calculations retain source precision; displayed values are rounded.

Execute displayed programs in order in a fresh Python process. Python 3.12.14; numpy 2.2.6; pandas 2.2.3; matplotlib 3.10.8; yfinance 1.2.0. Original snapshots and instructor checks remain private and are not distributed with the student version.